

Bab 14

Implementasi Multi-Bahasa

Tujuan Pembelajaran

Setelah mempelajari bab ini, mahasiswa mampu:

- Mengimplementasikan cipher dasar dalam MATLAB, C, C++, dan Pascal
- Membuat dokumentasi dan pengujian kode
- Membandingkan implementasi antar bahasa pemrograman

14.1 Implementasi dalam MATLAB

Berikut implementasi Caesar Cipher dan Algoritma Euclid Diperluas dalam MATLAB [1, 2].

14.1.1 Caesar Cipher

Listing 14.1: Implementasi Caesar Cipher dalam MATLAB

```
1 function ciphertext = caesar_encrypt(plaintext, shift)
2 % Caesar Cipher Encryption
3 % plaintext: string input
4 % shift: integer (0-25)
5     plaintext = upper(plaintext);
```

```

6     ciphertext = '';
7     for i = 1:length(plaintext)
8         if plaintext(i) >= 'A' && plaintext(i) <= 'Z'
9             c = double(plaintext(i)) - 65;
10            c = mod(c + shift, 26);
11            ciphertext = [ciphertext char(c + 65)];
12        else
13            ciphertext = [ciphertext plaintext(i)];
14        end
15    end
16 end

17
18 function plaintext = caesar_decrypt(ciphertext, shift)
19     plaintext = caesar_encrypt(ciphertext, 26 - shift);
20 end

```

14.1.2 Algoritma Euclid Diperluas

Listing 14.2: Extended Euclidean Algorithm dalam MATLAB

```

1 function [g, x, y] = extended_gcd(a, b)
2 % Extended Euclidean Algorithm: ax + by = gcd(a,b)
3     if b == 0
4         g = a;
5         x = 1;
6         y = 0;
7     else
8         [g, x1, y1] = extended_gcd(b, mod(a, b));
9         x = y1;
10        y = x1 - floor(a/b) * y1;
11    end
12 end
13
14 % Contoh: invers modular dari a mod m
15 function inv = mod_inverse(a, m)

```

```

16     [g, x, ~] = extended_gcd(mod(a, m), m);
17     if g ~= 1
18         inv = []; % tidak ada invers
19     else
20         inv = mod(x, m);
21     end
22 end

```

14.2 Implementasi dalam C

Berikut implementasi Caesar Cipher dan Fast Modular Exponentiation dalam C.

14.2.1 Caesar Cipher

Listing 14.3: Implementasi Caesar Cipher dalam C

```

1 #include <stdio.h>
2 #include <ctype.h>
3 #include <string.h>
4
5 void caesar_encrypt(char *plaintext, int shift) {
6     int i;
7     for (i = 0; plaintext[i] != '\0'; i++) {
8         if (isalpha(plaintext[i])) {
9             char base = isupper(plaintext[i]) ? 'A' : 'a';
10            plaintext[i] = base + (plaintext[i] - base + shift) %
11                           26;
12        }
13    }
14
15 void caesar_decrypt(char *ciphertext, int shift) {
16     caesar_encrypt(ciphertext, 26 - shift);
17 }
18

```

```
19 int main() {
20     char msg[] = "HELLO CRYPTO";
21     caesar_encrypt(msg, 3);
22     printf("Encrypted: %s\n", msg);
23     caesar_decrypt(msg, 3);
24     printf("Decrypted: %s\n", msg);
25     return 0;
26 }
```

14.2.2 Fast Modular Exponentiation

Listing 14.4: Fast Modular Exponentiation dalam C

```
1  #include <stdio.h>
2
3  long long mod_pow(long long base, long long exp, long long mod) {
4      long long result = 1;
5      base = base % mod;
6      while (exp > 0) {
7          if (exp % 2 == 1)
8              result = (result * base) % mod;
9          exp = exp / 2;
10         base = (base * base) % mod;
11     }
12     return result;
13 }
14
15 int main() {
16     printf("2^10 mod 1000 = %lld\n", mod_pow(2, 10, 1000));
17     printf("3^644 mod 645 (RSA) = %lld\n", mod_pow(3, 644, 645));
18     return 0;
19 }
```

14.3 Implementasi dalam C++

Berikut implementasi Caesar Cipher dalam C++.

Listing 14.5: Implementasi Caesar Cipher dalam C++

```
1 #include <iostream>
2 #include <string>
3 #include <cctype>
4
5 std::string caesar_encrypt(std::string plaintext, int shift) {
6     std::string ciphertext = "";
7     for (char c : plaintext) {
8         if (isalpha(c)) {
9             char base = isupper(c) ? 'A' : 'a';
10            ciphertext += base + (c - base + shift + 26) % 26;
11        } else {
12            ciphertext += c;
13        }
14    }
15    return ciphertext;
16 }
17
18 std::string caesar_decrypt(std::string ciphertext, int shift) {
19     return caesar_encrypt(ciphertext, 26 - shift);
20 }
21
22 int main() {
23     std::string msg = "HELLO CRYPTO";
24     std::cout << "Encrypted: " << caesar_encrypt(msg, 3) << std::
25         endl;
26     std::cout << "Decrypted: " << caesar_decrypt(
27         caesar_encrypt(msg, 3), 3) << std::endl;
28     return 0;
29 }
```

14.4 Implementasi dalam Pascal

Berikut implementasi Caesar Cipher dalam Pascal.

Listing 14.6: Implementasi Caesar Cipher dalam Pascal

```
1 program CaesarCipher;
2
3 var
4     plaintext, ciphertext: string;
5     shift, i: integer;
6     c: char;
7
8 function caesar_encrypt(s: string; k: integer): string;
9 var
10     result: string;
11     i: integer;
12     base: char;
13 begin
14     result := '';
15     for i := 1 to length(s) do
16     begin
17         if (s[i] >= 'A') and (s[i] <= 'Z') then
18         begin
19             base := 'A';
20             result := result + chr(ord(base) + (ord(s[i]) - ord(base) + k
21                 ) mod 26);
22         end
23         else if (s[i] >= 'a') and (s[i] <= 'z') then
24         begin
25             base := 'a';
26             result := result + chr(ord(base) + (ord(s[i]) - ord(base) + k
27                 ) mod 26);
28         end
29         else
30             result := result + s[i];
```

```
29     end;
30     caesar_encrypt := result;
31 end;
32
33 function caesar_decrypt(s: string; k: integer): string;
34 begin
35     caesar_decrypt := caesar_encrypt(s, 26 - k);
36 end;
37
38 begin
39     plaintext := 'HELLO CRYPTO';
40     shift := 3;
41     ciphertext := caesar_encrypt(plaintext, shift);
42     writeln('Encrypted: ', ciphertext);
43     writeln('Decrypted: ', caesar_decrypt(ciphertext, shift));
44 end.
```

Rangkuman

Bab ini menyajikan implementasi Caesar Cipher, Algoritma Euclid Diperluas, dan Fast Modular Exponentiation dalam empat bahasa pemrograman. Mahasiswa diharapkan dapat mengadaptasi kode untuk proyek implementasi algoritma kriptografi lainnya.

Bibliografi

- [1] Niels Ferguson, Bruce Schneier, and Tadayoshi Kohno. *Cryptography Engineering*. Wiley, 2010.
- [2] Bruce Schneier. *Applied Cryptography*. John Wiley & Sons, 2 edition, 1996.